

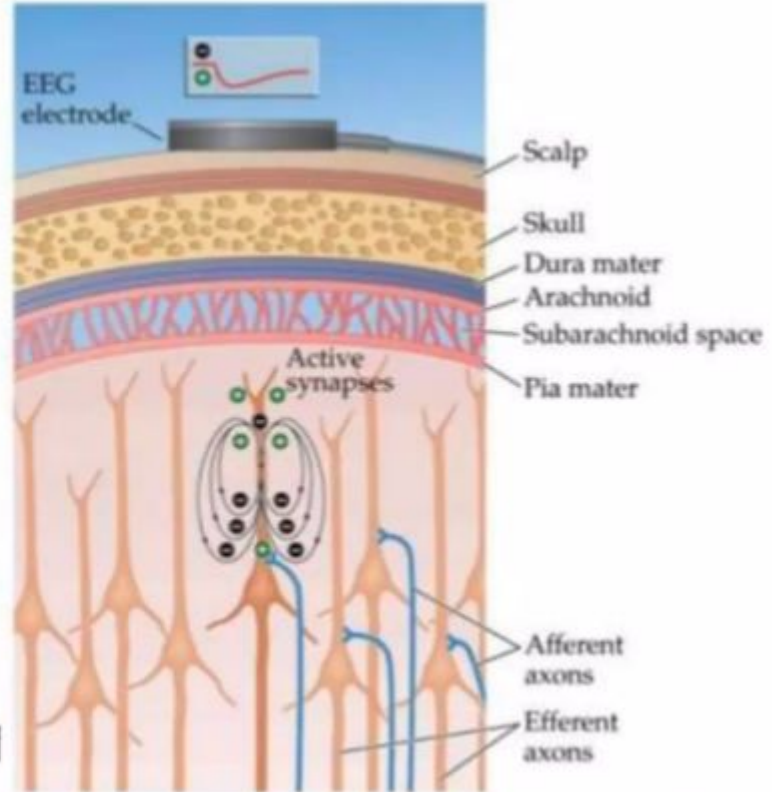


Deep Learning

Lectures on Advanced Topics & Case Studies
UG-4, Aug 2025-26
Shaik Mohammad Rafi B

Neuroscience

- EEG results from the *combined* activity of a *large* numbers of *similarly oriented* pyramidal neurons.
- Requires *synchronous* activity across groups of cells.
- EEG reflects *summed* post synaptic activity of large cell ensembles.

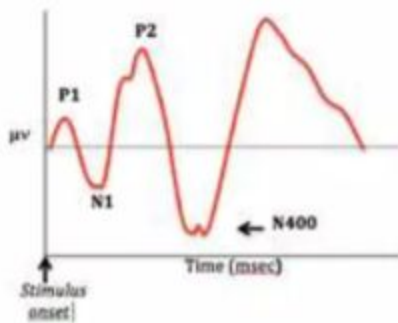




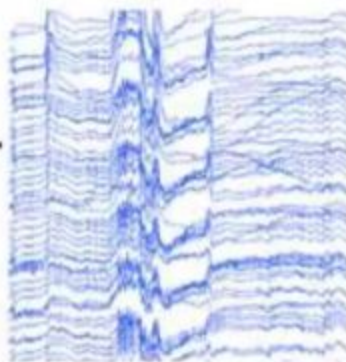
stimulus



amplifier

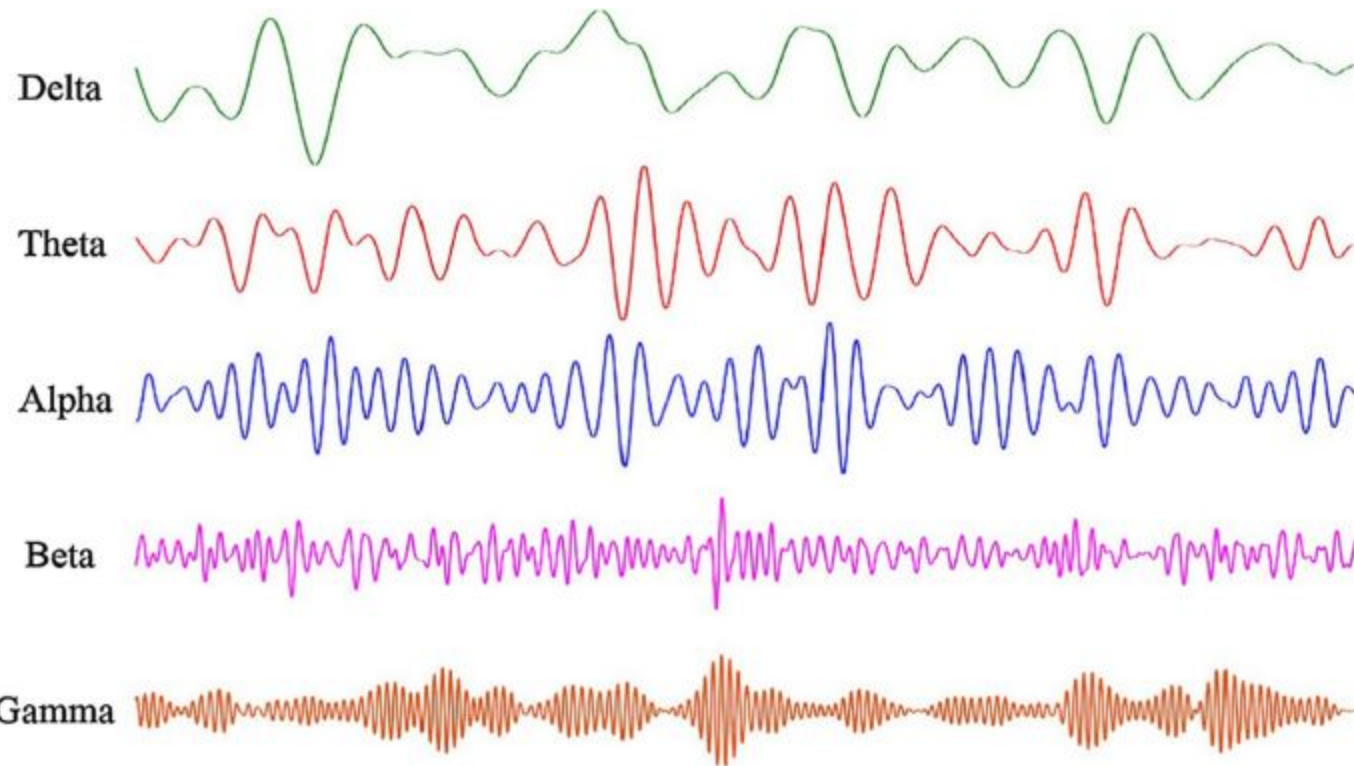


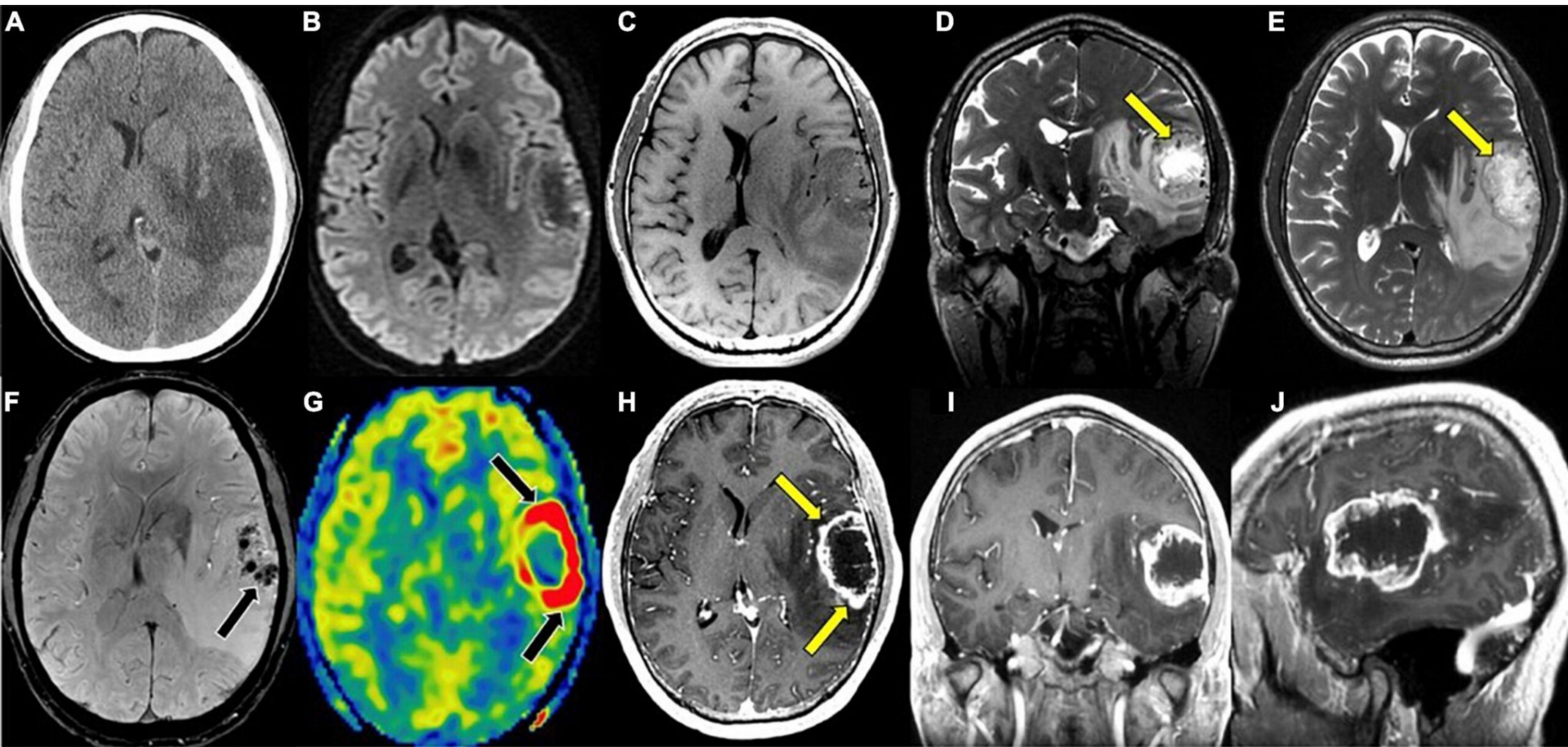
Cleaned, averaged
data

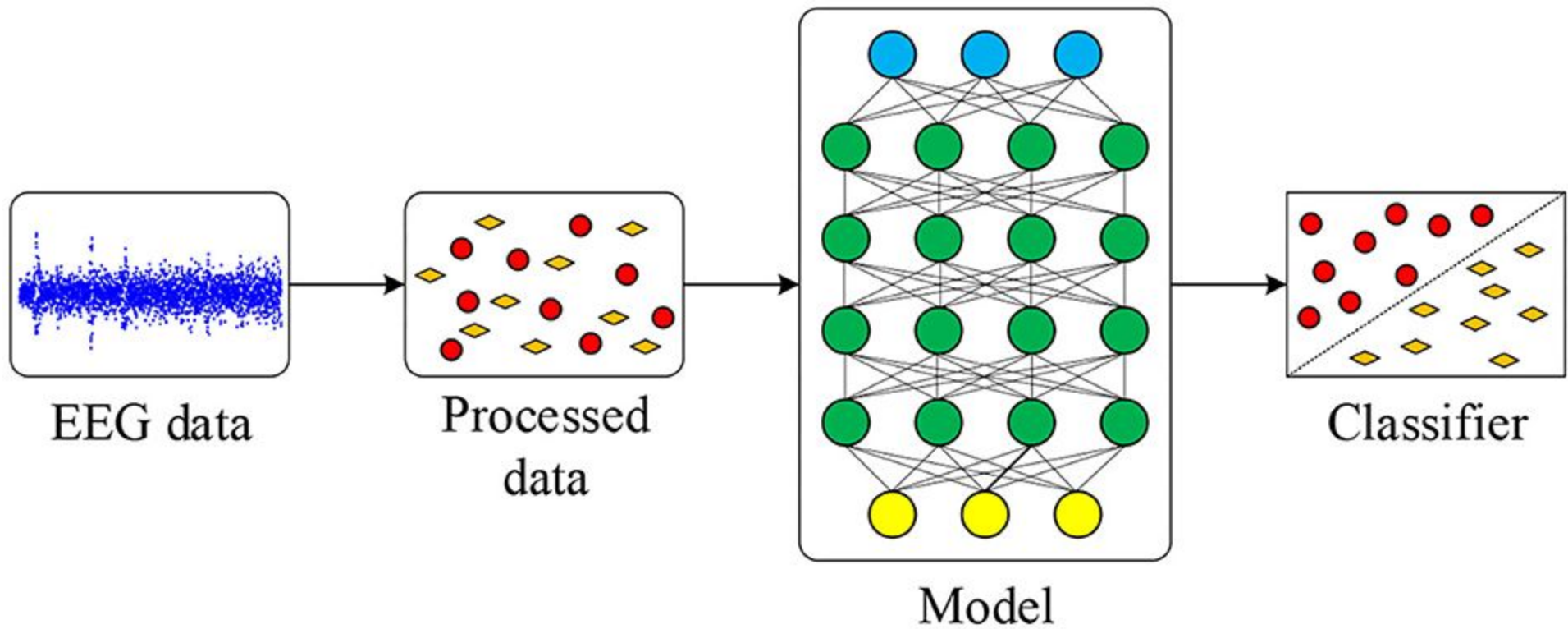


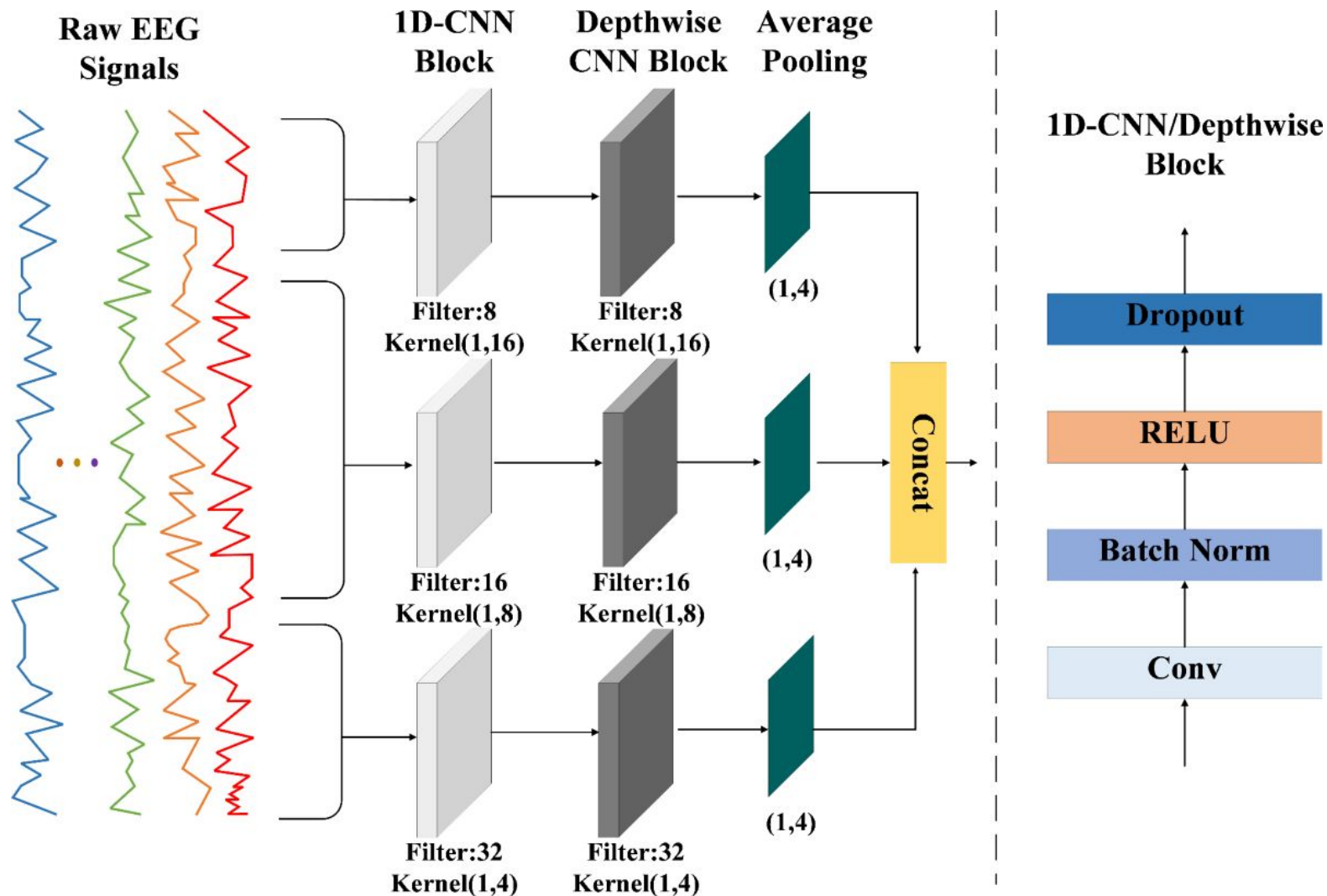
Example of a
printout of an
EEG recording
(raw data)











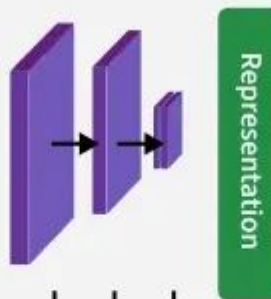
Self Supervised Learning

Self-Supervised Learning (SSL) is a type of machine learning where a model is trained using data that does not have any labels or answers provided.

Instead of needing people to label the data, the model finds patterns and creates its own labels from the data automatically.

Self Supervised Learning

Pretrain



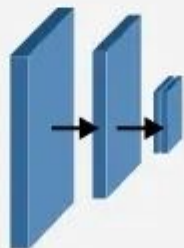
Representation



Pretext Task



Finetune

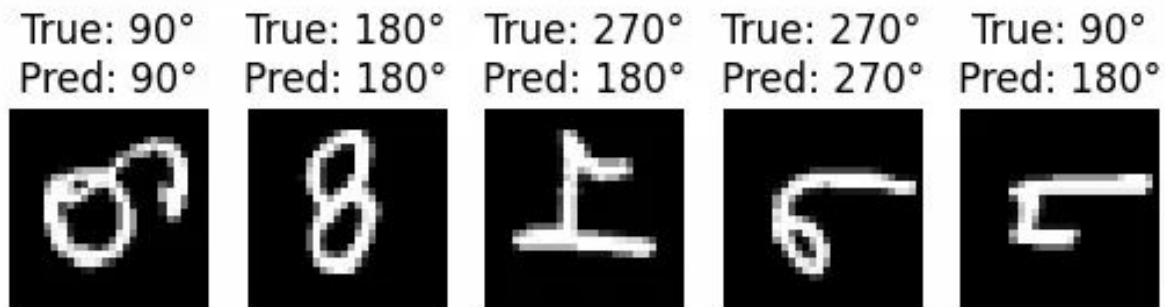


Representation

Classifier

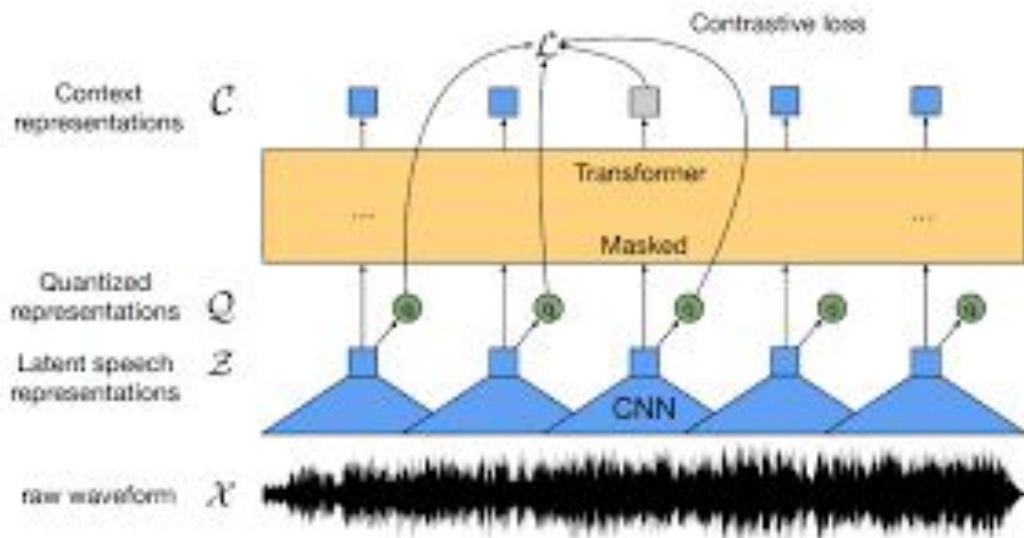


Dog



<https://www.geeksforgeeks.org/machine-learning/self-supervised-learning-ssl/>

wav2vec



Reinforcement Learning

Reinforcement learning (RL) is a type of machine learning where an **agent** learns to *make decisions* by interacting with an **environment** to maximize a cumulative **reward** signal.

Humans and animals learn through “trial and error” with feedback.

Ex: A dog learns to sit because it gets a treat; an RL agent learns to play chess because it gets a reward for winning.

Key components:

- Agent : the learner/decision-maker
- Environment: what the agent interacts with
- State: the current situation the agent observes
- Action: what the agent can do
- Reward: feedback signal (positive or negative) after each action
- Policy: the agent's strategy for choosing actions given a state

THE AGENT: The Chess Engine

THE ENVIRONMENT: The Chessboard and Rules

(Including squares, all pieces, and their legal movements)

REWARD:

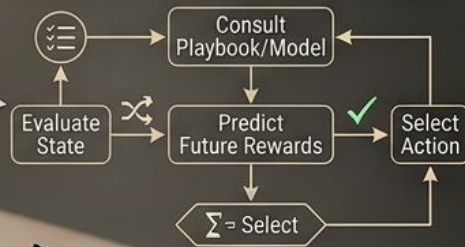
squares, all pieces, and their legal movements)

THE STATE: Current Board Configuration

(It captures the specific positions of all 32 pieces)



POLICY: The Learning Strategy



ACTION:
Moving Knight to g5



REWARD:
Immediate Feedback
(Positive/Negative)

THE AGENT:

The AI Driver Software

THE ENVIRONMENT:

The Road and traffic

(Including street markings, all objects, traffic lights, and pedestrians)

THE STATE:

Current Sensor Snapshot

(It captures the precise position of the car, other vehicles, lane lines, traffic light status, and pedestrian detection as seen by cameras and LiDAR, including distance data)



POLICY:

The Driving Strategy



POSITIVE:

- +50** Reach Destination
- +10** Stay in Lane
- +2** Safety Score

NEGATIVE:

- 5** Harsh Braking (!)
- 20** Traffic Violation 🚨
- 500** Collision Risk 🚗💥



ACTION:
Steering Left, Braking

REWARD SCORE:
Summation

Exploration and Exploitation in RL

Exploitation means sticking with what you already know is good.

In chess, this looks like: "Every time I reach this position, I play e4 because I've won with it before — so I'll keep doing it." The agent plays safe.

Exploration means trying something new, even at the risk of losing.

In chess, this looks like: "I've never tried the Sicilian Defense here — let me try it and see what happens, even though I might lose this game."

- If an RL agent only exploits, it gets stuck (trapped in a local optimum), never discovering that a completely different strategy is actually stronger.
- If an RL agent only explores, it never capitalizes on what it's learned. It keeps randomly trying wild moves and never builds up reliable winning strategies.

Trial & Error is generic (trying different foods in different restaurants)

Exploration and Exploitation is more specific (trying different restaurants and then different foods in best restaurants)

Learning Process

RL is formally defined as a Markov Decision Process (MDP), which is a mathematical framework for decision-making. An MDP is defined by a tuple (S, A, P, R, γ) :

- S — set of all possible *states*
- A — set of all possible *actions*
- $P(s' | s, a)$ — transition probability: chance of landing in state s' after taking action a in state s

MDP assumption: "The future depends only on the current state, not the history of how you got there."

$$P(s_{t+1} | s_0, a_0, \dots, s_t, a_t) \Rightarrow P(s_{t+1} | s_t, a_t)$$

- $R(s, a)$ — *reward* function: immediate reward for taking action a in state s
- γ — *discount factor* (between 0 and 1), how much future rewards are valued

Cumulative Reward

$$\begin{aligned} G_t &= R_{t+1} + \gamma \cdot R_{t+2} + \gamma^2 \cdot R_{t+3} + \dots \\ &= \sum \gamma^k \cdot R_{t+k+1} \end{aligned}$$

Policy (π): The agent's strategy is called a policy, mapping states to actions:

- Deterministic policy: $\pi(s) = a \rightarrow$ always take action a in state s
- Stochastic policy: $\pi(a | s) =$ probability of taking action a in state s

The goal of RL is to find the optimal policy π^ that maximizes cumulative reward.*

State-Value Function $V(s)$: Expected return starting from state s , following policy π

$$V^\pi(s) = E[G_t | s_t = s]$$

Action-Value Function $Q(s, a)$: Expected return starting from state s , taking action a , then following policy π

$$Q^\pi(s, a) = E[G_t | s_t = s, a_t = a]$$

$$= \sum_a \pi(a|s) \cdot \sum_{s'} P(s'|s, a) \cdot [R(s, a) + \gamma \cdot V^\pi(s')] \quad (\text{recursion})$$

immediate reward +
discounted value of the next state.

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) \cdot [R(s, a) + \gamma \cdot V^*(s')] \quad (\text{optimal state value})$$

$$Q^*(s, a) = \sum_{s'} P(s'|s, a) \cdot [R(s, a) + \gamma \cdot \max_{a'} Q^*(s', a')] \quad (\text{optimal Action Value})$$

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a) \quad (\text{optimal policy})$$

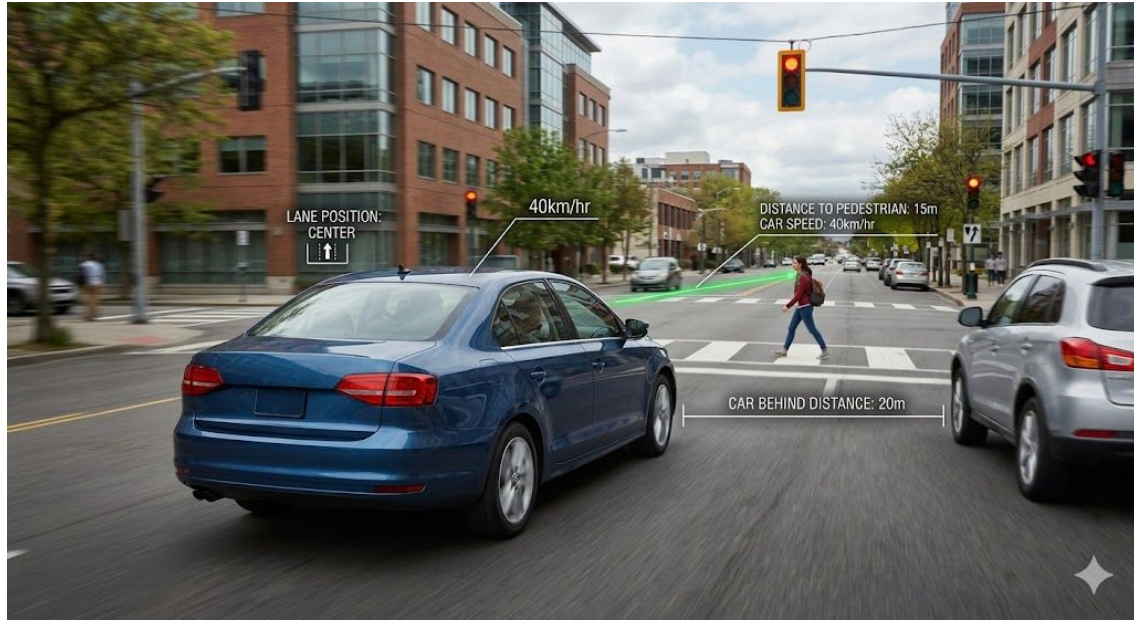
Driverless car case

Current state s

speed: 40 km/h,
distance_to_pedestrian: 15m,
traffic_light: RED,
car_behind_distance: 20m,
lane_position: center

Actions A:

brake_hard,
brake_gently,
maintain_speed,
accelerate,
steer_left,
steer_right



Transition Probabilities

$P(\text{safe_stop} \mid \text{current_state}, \text{brake_gently}) = 0.85$

$P(\text{too_close_to_pedestrian} \mid \text{current_state}, \text{brake_gently}) = 0.10$

$P(\text{rear_collision} \mid \text{current_state}, \text{brake_gently}) = 0.05$

Rewards

$R(\text{brake_gently at red light}) = +10 \leftarrow \text{safe, legal stop}$

$R(\text{accelerate through red}) = -100 \leftarrow \text{dangerous, illegal}$

$R(\text{brake_hard}) = -20 \leftarrow \text{safe but risks rear collision}$

$R(\text{hit pedestrian}) = -1000 \leftarrow \text{catastrophic}$

Cumulative reward

$G_t = R_{t+1} + \gamma \cdot R_{t+2} + \gamma^2 \cdot R_{t+3} + \dots$

$= (\text{brake gently}) + 0.95 \cdot (\text{stopped}) + 0.95^2 \cdot (\text{accelerate}) + \dots$

$= +10 + 0.95 \cdot (+5) + 0.9025 \cdot (+10)$

$= +10 + 4.75 + 9.02$

$= +23.77$

Cumulative reward

$G_t = R_{t+1} + \gamma \cdot R_{t+2} + \gamma^2 \cdot R_{t+3} + \dots$

$= (\text{accelerate}) + 0.95 \cdot (\text{accelerate}) + 0.95^2 \cdot (\text{brake hard}) + \dots$

$= -100 + 0.95 \cdot (-100) + 0.9025 \cdot (-20)$

$= \text{negative reward}$

$Q(\text{current_state}, \text{brake_gently}) = +23.77$

$Q(\text{current_state}, \text{accelerate}) = -200.00$

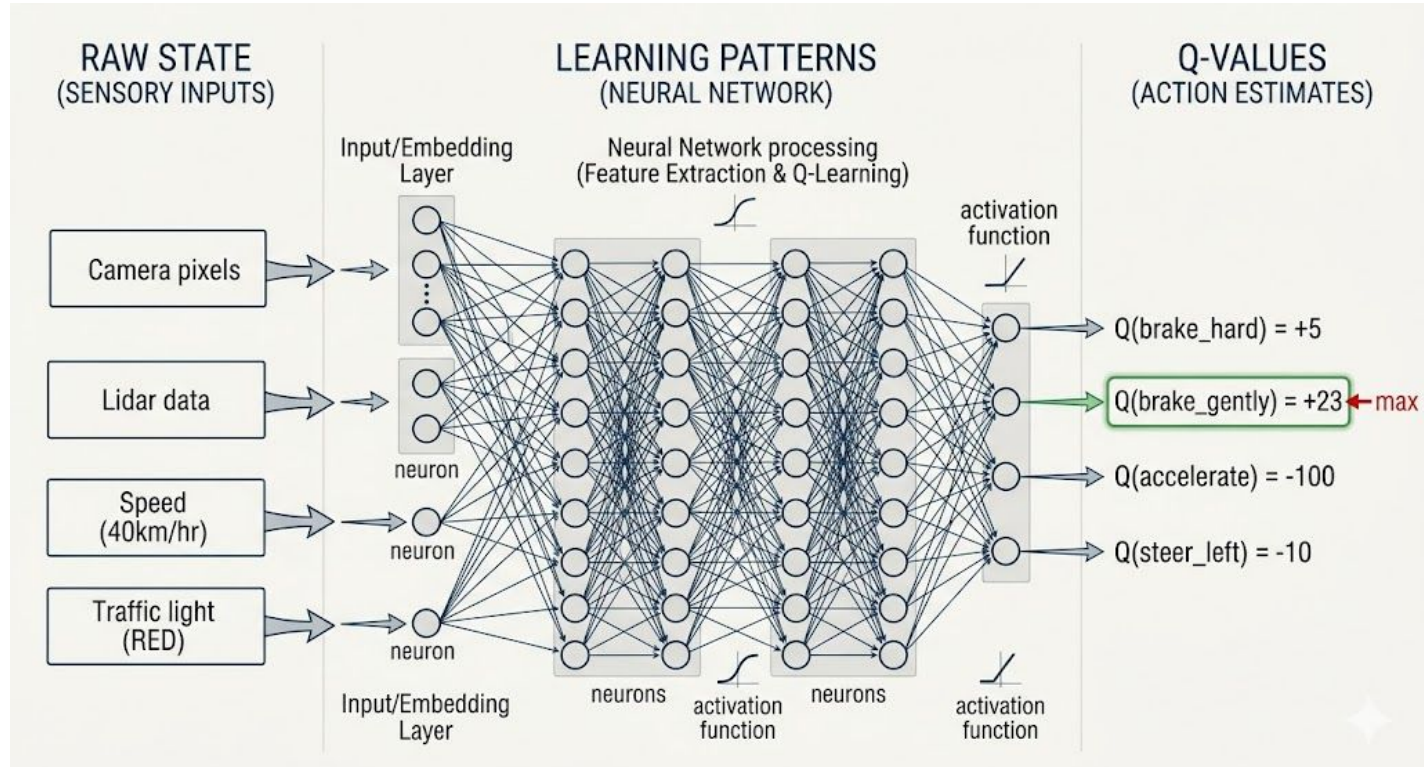
$Q(\text{current_state}, \text{brake_hard}) = +5.00$

$Q(\text{current_state}, \text{maintain_speed}) = -50.00$

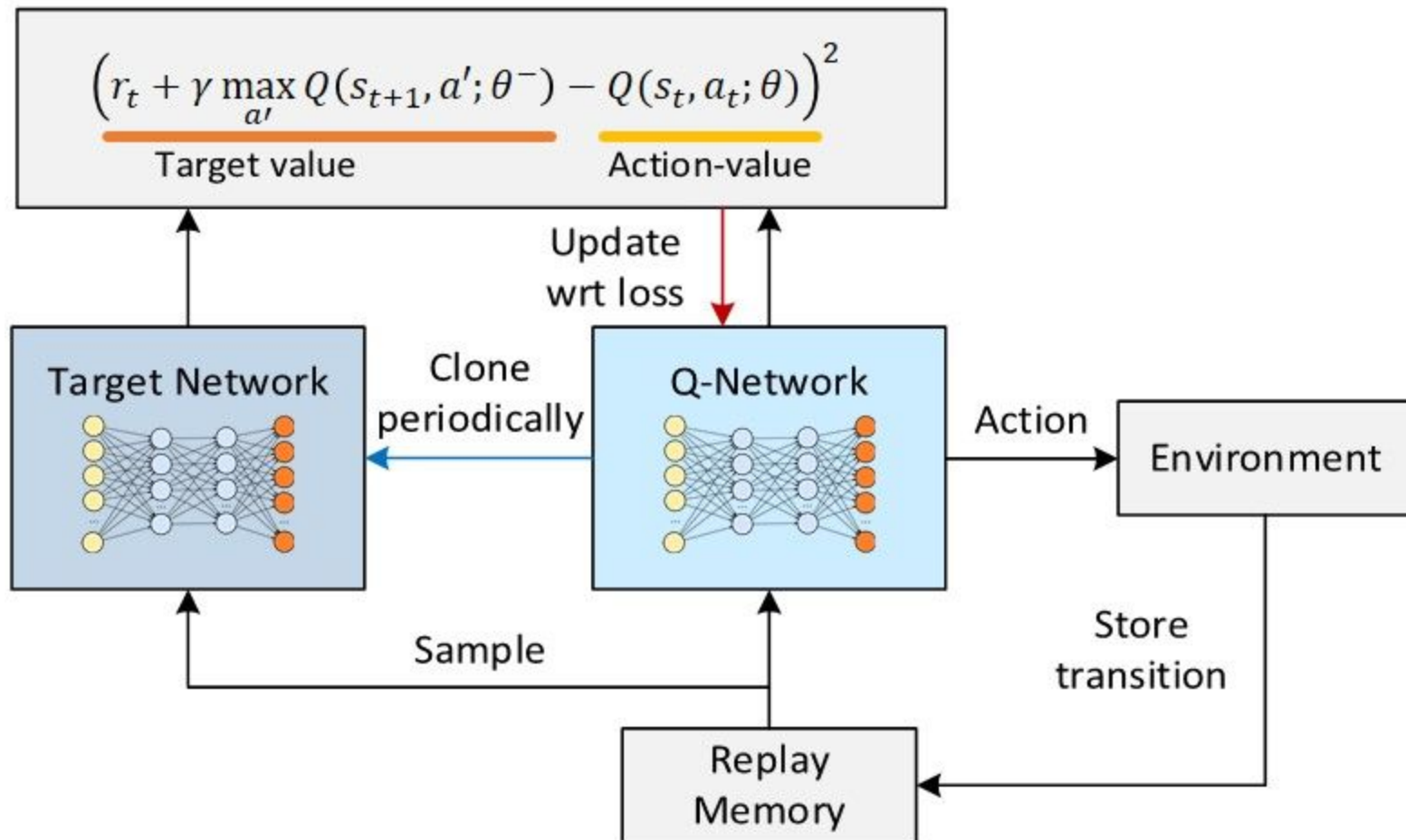
Optimal policy is to brake gently

Deep Reinforcement Learning

The network takes the raw state as input and outputs Q-values for all actions simultaneously.



DQN Loss



Deep RL

STEP 1 — OBSERVE

Current state $s = \{\text{red_light}, 15\text{m}, 40\text{km/h}\}$

STEP 2 — ACT (Online Network)

Online(s) $\rightarrow$ Q-values $\rightarrow$ argmax $\rightarrow$ brake_gently

STEP 3 — ENVIRONMENT RESPONDS

reward $R = +10$

new state $s' = \{\text{slowing}, 10\text{m}, 25\text{km/h}\}$

STEP 4 — COMPUTE TARGET (Target Network)

Target(s') $\rightarrow$ max Q(s') = 14.5

$y = 10 + 0.95 \times 14.5 = 23.77 \leftarrow$ STABLE TARGET

STEP 5 — COMPUTE LOSS (Online Network)

prediction = Online(s)[brake_gently] = 19.0

Loss = $\text{abs}(19.0 - 23.77) = 4.27$

STEP 6 — UPDATE ONLINE NETWORK

backpropagate loss $\rightarrow$ update θ

Online Network weights change

STEP 7 — TARGET NETWORK STAYS FROZEN

θ^- unchanged

STEP 8 — REPEAT 1000 TIMES

After 1000 steps:

$\theta^- = \theta \leftarrow$ copy online weights to target

Target Network now has updated knowledge

Freeze again for next 1000 steps

"Playing Atari with Deep Reinforcement Learning" (Mnih et al., 2013)

Input: Raw pixels from Atari screen ($210 \times 160 \times 3$)

Output: Which button to press (noob, fire, left, right...)

Reward: Game score

The Deep Q Network predicts Q values for each action from images

